

“FreeCut 视频”软件开发文档

1. 项目概述

FreeCut 是一个基于 Qt Quick 的多媒体视频编辑工具，支持视频剪辑、添加水印、画中画、合并视频等核心功能。项目采用 QML 构建用户界面，C++ 实现核心视频处理功能，通过 FFmpeg 进行底层视频操作。

- 1.基本功能：视频的导入，导出，剪切（剪切视频指可以任意截取一段视频并将此视频保存）
- 2.进阶功能：实现部分的剪辑功能（如 时间轴缩略图，多轨道，音频混合视频的合并等）

主要实现的功能：

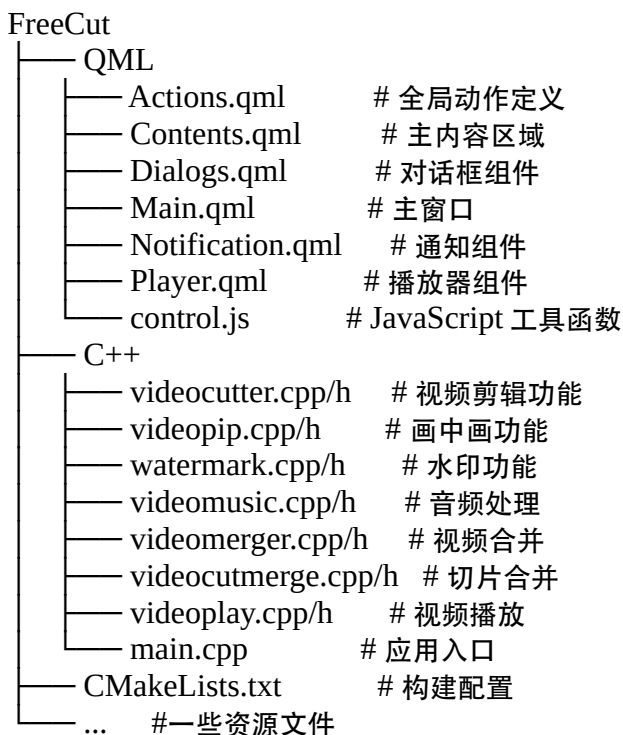
- 导入多种格式的视频/音频素材
- 视频剪辑（设置起始点/结束点，同一视频多次剪切，多次剪切的切片可以合并）
- 添加动态水印（可设置文本、颜色、大小、透明度）
- 画中画功能（支持位置拖拽）
- 视频合并（相同格式）
- 音频替换（关闭原声/插入新音频）
- 视频预览与播放控制

2. 技术架构

2.1 技术栈

- 前端: QML + JavaScript
- 后端: C++23 (Qt 6.9)
- 视频处理: FFmpeg
- 构建系统: CMake 3.31.1

2.2 软件编码结构



3. 核心模块设计

3.1 素材播放器模块

//-Player.qml-

// 文件选择对话框 FileDialog

//打开文件导入素材, `_mplay.source` 是设置播放视频是新添加的视频

```
FileDialog{
    id:_openfile
    ...
    nameFilters:["Video files (*.mp4 *.mov *.avi *.mkv *.mp3 *.wav *.flac *.ogg)"]
    onAccepted: {
        _mplay.source = selectedFile
        inputPath = selectedFile.toString().replace("file://", "")
    }
}
```

//-Controller-

//区分添加的素材是音频还是视频

```
function detectMediaType(path) {
    var videoExts = ["mp4", "mov", "avi", "mkv"]; // 视频类型
    var audioExts = ["mp3", "wav", "flac", "ogg"]; // 音频类型
    if (videoExts.includes(splitPath)) return "video";
    if (audioExts.includes(splitPath)) return "audio";

    return "other";
}
```

//-Contents.qml-

//_dialog 中使用 FileDialog,存储导入的视频路径等信息到 ListModel 中

```
ListModel{
    id:videoModel
}
//videoModel dialog openfiles:
Player{
    id:_dialog
    openfile{
        ...
        onAccepted: {
            let selectfilePath = openfile.selectedFiles;
            for(let i = 0; i < selectfilePath.length; ++i){
                const filePath = selectfilePath[i]
                _mplayer.source = filePath
                let mediaTypeFlag = Controller.detectMediaType(filePath); //区分是视频还是音频的 JS
                const fileName = filePath.toString().split('/').pop().replace(/^[^\.]+\$/, "")
                videoModel.append({
                    title: fileName,
                    filePath: filePath,
                    mediaType: mediaTypeFlag
                })
            }
        }
    }
}
```

函数

// 素材列表展示

ListView {

model: videoModel

delegate: Video {

source: model.filePath

//文本显示：素材音频的标题，这里考虑用 Loader 是因为 JS 文件和 qml 文件的异步问题，但如果
没有 active: model.mediaType === "audio"就不用 Loader

Loader{

// active: model.mediaType === "audio"

sourceComponent: Component{

Text{

id:audioText

text:model.title

}

}

}

//视频第一帧显示：这里巧妙的让视频播放 0.1 秒就暂停，就可以实现第一帧

Video {

autoPlay: true

muted: true

loops: MediaPlayer.Infinite

onPlaybackStateChanged: {

seek(100)

pause()

}

}

}

}

qml 组件中的素材播放器，以及直接观看剪辑好的视频的播放器，首先用 ListView+ListModel，将选择的素材竖直排列在左上的区域内,点击素材可以在播放器中播放该素材视频。

通过剪辑好的视频的保存路径，可以将其添加进下方区域的 ListView 中，封面是视频的第一帧画面，或者音频的标题。点击素材封面可以通过 videoplay 播放器弹出一个窗口播放剪辑好的视频，封面下方也会显示该视频的路径，方便客户找到视频。

3.2 预览窗口模块

//初始化状态

property bool showSelectButton: true // 初始显示"添加素材"按钮

property bool isVideoPlaying: false // 初始无视频播放

//点击"添加素材"按钮：

Button {

id: select

onClicked: dialog.open() // 打开素材选择对话框

// 当按钮尺寸变化时，重新计算字体大小

onWidthChanged: text.font.pixelSize = Controller.calculateFontSize(select)

onHeightChanged: text.font.pixelSize = Controller.calculateFontSize(select)

}

//素材选择对话框：

Dialog {

id: dialog

ListView {

model: videoModel // 显示所有导入的素材

delegate: Item {

```

    TapHandler {
        onPressed: {
            oneplayer.mplay.source = model.filePath
            oneplayer.mplay.play() // 播放选中素材
            rightRect.showSelectButton = false // 隐藏添加按钮
        }
    }
}
}
}
}

```

//时间轴同步

```

Slider {
    id: slider2
    value: oneplayer.mplay.position // 绑定播放位置
    onValueChanged: {
        if (pressed) {
            oneplayer.mplay.position = value // 拖动跳转
        }
    }
}
}

```

// 时间显示, `formatTime` 是时间转换, 写在 Controller 是优化直接在.qml 中多次重复编写

```

Text {
    text: Controller.formatTime(oneplayer.mplay.position) + "/" +
        Controller.formatTime(oneplayer.mplay.duration)
}

```

//-Controller-

// 格式化时间为 mm:ss

```

function formatTime(milliseconds) {
    var seconds = milliseconds / 1000;
    var minutes = Math.floor(seconds / 60);
    seconds = Math.floor(seconds % 60);
    return minutes + ":" + (seconds < 10 ? "0" : "") + seconds;
}

```

// 播放状态同步

```

MediaPlayer {
    onPlayingChanged: {
        if(playing) {
            // 更新界面状态
            rightRect.isVideoPlaying = true
        }
    }
}
}

```

// 素材变化同步, `Connections` 依然是为了能够在 cpp 后端处理完视频后才执行 qml 代码, 即 c++ 和 qml 的异步问题, `cutter` 是 `videocutter` 的后端函数作为 qml 的一个类型实例化的 id, `oneplayer` 是预览窗口实例化的一个 `Player` 类型

```

Connections {
    target: cutter
    onCutFinished: {
        if(success) {

```

```

        oneplayer.mplay.source = "" // 重置播放器
        oneplayer.mplay.source = "file://" + outputPath // 加载新视频
    }
}

```

用户通过点击窗口中的添加素材会弹出一个素材选择的 List,即素材导入中的 List,然后就可以播放视频来预览效果,当然最初播放的视频是没有经过处理的原视频,和素材列表中的视频一致。

3.3 剪切功能

//后端 c++类型实例化

```

VideoCutter {
    id: cutter
    onFinished: (success, error) => {
        console.log("剪切结果:", success, error)
        noti.text = success ? "剪切成功!" : "失败: " + error
        noti.notification.show(noti.text)
    }
}

//单次剪切 workflow
//标记起始点
Button {
    // 格式化时间为秒数
    function formatTime(milliseconds) {
        return Math.floor(milliseconds / 1000); // 返回秒数
    }
    onClicked: {
        cutter.getStartSec(formatTime(oneplayer.mplay.position));
    }
}

//标记结束点
Button {
    // 格式化时间为秒数
    function formatTime(milliseconds) {
        return Math.floor(milliseconds / 1000); // 返回秒数
    }
    onClicked: {
        cutter.getEndSec(formatTime(oneplayer.mplay.position));
    }
}

//预览剪辑
Button {
    onClicked: {
        // 调用预览功能, c++函数, 文件的输入路径, 开始时间和结束时间
        cutter.previewCut(inputPathPreview, cutter.returnStartSec(), cutter.returnEndSec())
    }
}

//保存
FileDialog {
    onAccepted: {
        // 执行实际剪切
        cutter.cutVideo(inputPath, outputPath, startSec, endSec-startSec);
    }
}

```

```
}
```

//多次剪切(多剪切)实现

多剪切允许用户连续多次剪切并最终得到最终剪切的视频

// 控制状态

property int currentStep: 1 // 1-开始 2-标记起点 3-标记终点 4-保存/继续

Button {

text: "单击保存 | 双击继续剪切"

onClicked: {

isDoubleClicked=false

clickTimer1.start()

}

onDoubleClicked: {

isDoubleClicked=true

resultText.text = "已退出剪切"

messageTimer1.start()

currentButton = 1

Controller.deletefile("/root/wawawawawa")//删除临时文件

}

//处理单击继续剪切操作

Timer{

id:clickTimer1

interval: 300 // 300 毫秒是双击检测的合理时间

onTriggered:{

if(button6.isDoubleClicked===false){

//保存到固定的目录，保存剪切的视频并继续剪切，设置临时目录为 wawa 和

临时文件 cccValue

var outputPath = "/root/wawawawawa/ccc"+timerValue+".mp4"

Controller.processCut(inputPathPreview,outputPath)

inputPathPreview = outputPath

currentButton = 1

timerValue++

}

}

}

//连接 c++后端函数槽，timer //播放定时器（解决立即播放问题）

Connections {

target: cutter

function onCutFinished(success, outputPath) {

if(success) {

console.log("剪切完成，准备播放:", outputPath)

// 添加延迟器确保播放器状态重置

playTimer.outputPath = outputPath

playTimer.start()

} else {

resultText.text = "剪切失败"

resultText.color = "red"

messageTimer1.start()

```

    }
}

}

}

//视频剪切核心方法: c++调用 ffmpeg 命令行
//--cpp-
void VideoCutter::cutVideo(const QString &inputPath,
                           const QString &outputPath,
                           qint64 startSec,
                           qint64 durationSec)
{
    if (m_process) {
        m_process->kill();
        m_process->deleteLater();
        m_process = nullptr; // 重置指针
    }

    m_process = new QProcess(this);
    m_durationSec = durationSec;
    // 构建 FFmpeg 命令
    QStringList args = {"-y",
                        "-ss",
                        QString::number(startSec),
                        "-i",
                        inputPath,
                        "-t",
                        QString::number(durationSec),
                        "-c",
                        "copy",
                        outputPath};

    m_process->start("ffmpeg", args);
    m_process->waitForFinished();

    // 处理结果并发送信号以便于 qml 中用箭头函数可以或者信息
    connect(m_process, &QProcess::finished, this, [=, this](int exitCode, QProcess::ExitStatus) {
        QString result = (exitCode == 0) ? "成功" : "失败";
        qDebug() << "FFmpeg 进程结束:" << result << ", 退出码:" << exitCode;
        emit finished(exitCode == 0, m_process->errorString());
    });

    connect(m_process, &QProcess::finished, this, [=, this](int exitCode, QProcess::ExitStatus) {
        bool success = (exitCode == 0);
        QString resultMsg = success ? "成功" : "失败";
        qDebug() << "FFmpeg 剪切完成:" << resultMsg;
        emit cutFinished(success, outputPath); // 发射信号包含输出路径
    });
}

```

预览功能实现:

//管道: ffmpegplay & ffmpeg, 但不能循环播放,使用管道技术就可以不用产生临时文件, 也是预览的另一种实现方式: 弹出口

```

void VideoCutter::previewCut(const QString &inputPath, qint64 startSec, qint64 endSec)
{
    // 计算持续时间 (秒)
    qint64 durationSec = endSec - startSec;
    // 创建播放进程
    QProcess *player = new QProcess();

    // 设置环境变量
    QProcessEnvironment env = QProcessEnvironment::systemEnvironment();
    env.insert("PATH", "/usr/local/bin:/usr/bin:/bin");
    env.insert("LD_LIBRARY_PATH", "/usr/local/lib:/usr/lib");

    // 构建 FFmpeg 命令
    QProcess *ffmpegProcess = new QProcess();
    QStringList ffmpegArgs;
    ffmpegArgs << "-ss" << QString::number(startSec) << "-to" << QString::number(endSec) << "-i"
    << inputPath << "-c"
        << "copy"
        << "-f" << "matroska"
        << "-"; // 输出到 stdout

    // 构建 FFplay 命令
    QStringList ffplayArgs;
    ffplayArgs << "-autoexit"
        << "-window_title" << "视频预览"
        << "-i" << "pipe:0"; // 从 stdin 读取
    // 设置环境并连接进程
    ffmpegProcess->setProcessEnvironment(env);
    player->setProcessEnvironment(env);

    // 关键修复：正确连接进程管道
    ffmpegProcess->setStandardOutputProcess(player); // 使用正确的函数名
    player->setProcessChannelMode(QProcess::ForwardedChannels);

    // 先启动播放器，再启动 FFmpeg
    player->start("ffplay", ffplayArgs);
    ffmpegProcess->start("ffmpeg", ffmpegArgs);
}

```

单开两个线程，使用内存管道避免临时文件，FFmpeg → FFplay 直接流传输，注意 ffplay 和 ffmpeg 之间的顺序问题，实现实时解码播放。

3.4 原声开关+替换音频

//通过控制`MediaPlayer`的`muted`属性实现音频开关：

```

MediaPlayer {
    id: player
    audioOutput: AudioOutput {
        id: audioOutput
        muted: false // 默认开启原声
    }
}

```


// 原声开关按钮，使用 **RadioButton**

```
Row {
  RadioButton {
    text: "关闭原声"
    onClicked: {
      oneplayer.mplay.audioOutput.muted = true
      showNotification("已关闭原声");
    }
  }
  RadioButton {
    text: "打开原声"
    onClicked: {
      oneplayer.mplay.audioOutput.muted = false;
      showNotification("已打开原声");
    }
  }
}
```

实时切换：切换时立即生效，无需重新加载视频，且保持视频播放进度不变状态同步：

```
// 当视频切换时自动重置音频状态
onSourceChanged: audioOutput.muted = false
```

// 音频替换实现

// 工作流程：

1. 用户选择新音频文件
2. 系统提取视频流+新音频流
3. 合成新视频文件
4. 替换当前播放内容

// 保存替换了音频的视频

```
FileDialog{
  id: _savevideofile
  title: "Save your cut video"
  nameFilters: [ "Audio files (*.mp4 *.mov *.avi *.mkv *.wav)" ]
  onAccepted: {
    outputPath = selectedFile.toString().replace("file://", "")
    const fileName1 = selectedFile.toString().split('/').pop().replace(/\.[^/.]+$/, "")
    vimu.replaceAudio(content.inputPathPreview, audioOutputPath, outputPath)
  }
}
```

// **videomusic.cpp**

// 替换视频中的音频

```
bool VideoMusic::replaceAudio(const QString& videoPath, const QString& audioPath, const
QString& outputPath)
{
  ffmpeg = new QProcess(this);
  ffmpeg->start("ffmpeg",
    {"-y",
     "-i",
     videoPath,
     "-i",
     audioPath,
```

```

        "-c:v",
        "copy",
        "-map",
        "0:v:0",
        "-map",
        "1:a:0",
        "-shortest",
        outputPath});
return ffmpeg->waitForFinished() && ffmpeg->exitCode() == 0;
}

```

"-c:v", "copy" // 直接复制视频流

避免视频重编码，保持原始质量，处理速度提升

"-shortest" // 以最短的输入流结束

自动适配视频/音频时长差异，避免音视频不同步问题，处理不同长度的音视频。

原声控制 → 音频选择 → 流处理 → 新视频生成 → 无缝切换

videomusic 是一个 c++ 实现后端，通过 ffmpeg 调用命令行，实现可以将视频的音频替换为另一个音频（此音频可以为 mp3 这种纯音频，也可以是某个视频的音频），主要是将选中视频的视频流和音频的音频流合并为一个新的视频并保存，在右上方的播放器中通过 mediaplayer 可以实现关闭原声和打开原声，可以同步播放视频和另一个音频，点击确定可以将该音频插入视频中（但好像不能保存原声，涉及音频的编码，计划在暑假实现编码）

3.5 合并+合并切片(临时文件管理)

用户界面(QML) → 控制逻辑(JavaScript) → 核心处理(C++/FFmpeg)

↑
临时文件管理

双视频合并实现

1. 用户选择视频（当前预览窗口的列表）
2. 指定输出路径
3. 调用 FFmpeg 合并视频

// 选择视频（当前预览视频窗口）

```

Button {
    onClicked: {
        mergePath1 = oneplayer.mplay.source.toString().replace("file://", "")//第一个要合并的视频的路径
        dialog.open(); // 打开素材选择对话框
    }
}

```

// 执行合并

```

Button {
    id: mergeButton
    enabled: false
    onClicked: {
        oneplayer.savemergerfile.open()
    }
}

```

// 保存对话框

```

FileDialog {

```

```

onAccepted: {
    const outputPath = selectedFile.toString().replace("file://", "");
    videoMerger.mergeVideos(mergePath1, mergePath2, outputPath);
}
}
//videomerger.cpp
void VideoMerger::mergeVideos(const QString& inputPath1,
                              const QString& inputPath2,
                              const QString& outputPath)
{
    // 构建 FFmpeg 命令
    QStringList args = {
        "-y",
        "-i", inputPath1,
        "-i", inputPath2,
        "-filter_complex",
        "[0:v:0][0:a:0][1:v:0][1:a:0]concat=n=2:v=1:a=1[outv][outa]",
        "-map", "[outv]",
        "-map", "[outa]",
        outputPath
    };
}

```

切片合并实现

工作流程:

1. 用户标记多个片段 (开始/结束点)
2. 生成临时切片文件
3. 创建合并列表文件
4. 合并所有切片

// 切片 workflow 控制

property int mergeState: 0 // 0-未开始 1-标记起点 2-标记终点 3-确认片段

```

Button {
    text: "开始剪切"
    onClicked: {
        // 记录开始时间
        cutter.getStartSec(formatTime(oneplayer.mplay.position));
        mergeState = 2; // 进入标记结束状态
    }
}

```

```

Button {
    text: "结束剪切"
    enabled: mergeState === 2
    onClicked: {
        // 记录结束时间
        endSec = Math.floor(player.position/1000);
        mergeState = 3; // 进入确认状态
        cutter.getEndSec(formatTime(oneplayer.mplay.position));
        file = Controller.returnpath()
    }
}

```

```

        file1 = Controller.returnfilepath() //保存切片视频路径
        vcm.writepath(file)
    }
}

Button {
    text: "确认片段"
    enabled: mergeState === 3
    onClicked: {
        cutter.cutVideo1(filecut,file1,cutter.returnStartSec(),cutter.returnEndSec()-
cutter.returnStartSec()) //剪切
        mergeState = 1; // 回到标记开始状态
    }
}

Button {
    text: "合并"
    onClicked: {
        oneplayer.saveCutmerge.open()
    }
}
//合并切片
Rectangle{
    id:mergecut
    Button{
        id:mergebutton
        text:qsTr("合并切片")
        onClicked: {
            vcm.creadtefile()
        }
    }
}
}

// 自动管理临时目录
自动创建唯一临时目录,程序退出时自动清理, 避免硬盘空间浪费
bool VideoCutMerge::creadtefile()
{
    QString str = QString::number(a);
    tempFilePath = tempDirPath + "/tempfile" + str + ".txt";
    a++;
    QFile file(tempFilePath);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qCritical() << "无法创建文件";
        return 1;
    }
    file.close();
    return true;
}
//videocutmerge.h
~VideoCutMerge()
{
    // 析构时删除临时文件
    // tempFile.remove();
}

```

```

// 3. 确保程序退出时删除目录（即使发生异常）
auto cleanup = qScopeGuard([&] {
    QDir dir(tempDirPath);
    if (dir.exists()) {
        dir.removeRecursively();
        qDebug() << "已清理临时目录";
    }
});
}
//videocutmerge.cpp
bool VideoCutMerge::mergeVideos(const QString &outputPath)
{
    QFile file(tempFilePath);

    if (!QFile::exists(tempFilePath)) {
        qCritical() << "列表文件不存在";
        return false;
    }
    // 确保文件内容已写入磁盘
    file.close();

    // FFmpeg 命令
    QStringList args;
    args << "-f" << "concat"
        << "-safe" << "0"
        << "-i" << tempFilePath << "-c" << "copy" << outputPath;
    QProcess ffmpeg;
    ffmpeg.start("ffmpeg", args);

    return true;
}

```

功能	双视频合并	切片合并
输入	两个完整视频文件	单个视频的多个片段
处理方式	首尾衔接	按标记点提取片段后拼接
核心技术	直接调用命令行	临时文件列表+自动清理临时文件
输出	两个视频合并结果	精选片段合集
优势	简单快速	精确控制内容
适用场景	合并完整视频	创建精彩集锦/删除无效片段

videomeger 是通过 Qpreoss 调用 ffmpeg 命令行将两个视频合并在一起，但是因为视频编码格式不同的原因，现在只能合并视频和该视频的切片，编码格式不一样或者两种格式的视频都不能合并，涉及视频转码，之前试过用 ffmpeg 转码，但是转码后的视频不能播放或者也不可以合并，涉及 ffmpeg 底层编码转码（计划在暑假学习）

videocutmeger 是在项目目录下创建一个临时目录，选择一个视频，将视频的切片保存在临时目录下，并把路径保存进自动创建的.txt 文件中，通过 Qperocess 调用 ffmpeg 命令行将切片合并为一个新的视频并保存

3.6 水印

工作流程

1. 用户打开水印设置对话框
2. 配置水印参数（文本、颜色、大小、透明度）
3. 点击确定后选择输出路径
4. 调用 FFmpeg 添加水印
5. 保存并预览结果

水印参数设置界面(QML)

// 水印设置对话框

```
Popup {
    id: inputDialog
    ColumnLayout {
        // 水印文本输入
        TextField {
            id: textInput
            placeholderText: "输入水印文本"
        }

        // 颜色选择
        RowLayout {
            Text { text: "颜色:" }
            ComboBox {
                id: colorInput
                model: ["white", "red", "green", "blue", "black", "purple"]
            }
        }

        // 透明度设置
        RowLayout {
            Text { text: "透明度:" }
            ComboBox {
                id: alphaInput
                model: ["0", "0.2", "0.4", "0.6", "0.8", "1"]
            }
        }

        // 字体大小设置
        RowLayout {
            Text { text: "字体大小:" }
            SpinBox {
                id: sizeInput
                from: 10
                to: 72
                value: 24
            }
        }
    }

    Button {
        text: qsTr("确定")
        onClicked: {
            var customText = textInput.text;
            var customColor = colorInput.currentText;
            var customSize = sizeInput.value;
            var customAlpha = parseFloat(alphaInput.currentText);
```

```

        // 将参数传递给 oneplayer.watermark
        oneplayer.watermark.text = customText;
        oneplayer.watermark.color = customColor;
        oneplayer.watermark.size = customSize;
        oneplayer.watermark.alpha = customAlpha;

        inputDialog.close();
        //选择文件路径保存加上水印的视频
        //保存的时候将输出路径传给添加水印的函数，将水印添加到视频中
        oneplayer.savewater.open();
    }
}
}

```

水印处理命令

```

// watermark.cpp
void Watermark::addTextWatermark(const QString &inputPath,
                                  const QString &outputPath,
                                  const QString &text,
                                  const QString &color,
                                  int fontSize,
                                  double opacity)
{
    // 构建 FFmpeg 命令
    QString filter = QString("drawtext=text='%1':fontcolor=%2@%3:fontsize=%4:x=20:y=20")
        .arg(text)
        .arg(color)
        .arg(opacity)
        .arg(fontSize);

    QStringList args = {
        "-y",
        "-i", inputPath,
        "-vf", filter,
        "-c:a", "copy",
        outputPath
    };

    // 执行命令
    QProcess ffmpeg;
    ffmpeg.start("ffmpeg", args);
}

```

水印位置处理(C++)

```

// watermark.cpp
void Watermark::setPosition(int x, int y)
{
    // 保存位置信息用于 FFmpeg 命令
    positionX = x;
    positionY = y;
}

```

QString Watermark::generateFilter() const

```

{
    return QString("drawtext=text='%1':fontcolor=%2@%3:fontsize=%4:x=%5:y=%6")
        .arg(text)
        .arg(color)
        .arg(opacity)
        .arg(fontSize)
        .arg(positionX)
        .arg(positionY);
}

```

//使用 QPainter 在指定的位置绘制水印文本。

```

void Watermark::paint(QPainter *painter)
{
    //设置颜色
    painter->setPen(m_color);
    // 设置字体和大小
    QFont font = painter->font();
    font.setPointSize(m_size);
    painter->setFont(font);
    // 绘制文本
    painter->drawText(m_position, m_text); //在 m_position 位置绘制 m_text 文本
}

```

//更新水印位置绘制出水印动态的效果

```

Watermark::Watermark(QQuickItem *parent) : QQuickPaintedItem(parent)
{
    connect(&m_timer, &QTimer::timeout, this, [this]() {
        // 更新水印位置
        m_position.setX(m_position.x() + m_directionX);
        m_position.setY(m_position.y() + m_directionY);

        // 边界检查
        if (m_position.x() >= width() || m_position.x() <= 0) {
            m_directionX *= -1; // 反转 x 方向
        }
        if (m_position.y() >= height() || m_position.y() <= 0) {
            m_directionY *= -1; // 反转 y 方向
        }
        update(); // 触发重绘
    });
    m_timer.start(50); // 每 50 毫秒更新一次
}

```

水印参数变化实时更新，坐标自动映射，但实现的功能水印智能不规则移动

3.7 画中画

//预览窗口处理画中画分辨率和拖动范围

```

VideoOutput{
    id:out
    property double videoWidthPip: 0.0 //画中画，背景视频的原本宽度
    property double videoHeightPip: 0.0 //画中画，背景视频的原本高度
    property double videoXPip: 0.0 //画中画，背景视频的起点 x
    property double videoYPip: 0.0 //画中画，背景视频的起点 y
}

```



```

//视频实际绘制区域的变化
onContentRectChanged: {
    //获取画中画所需数值
    videoWidthPip = contentRect.width
    videoHeightPip = contentRect.height
    videoXPip = contentRect.x
    videoYPip = contentRect.y
}

function drag(addVideo){
    addVideo.x = Math.max(videoXPip,Math.min(addVideo.x, videoWidthPip-addVideo.width))
    addVideo.y = Math.max(videoYPip,Math.min(addVideo.y, videoHeightPip-
addVideo.height))
    console.log("x,y drag: ",addVideo.x,addVideo.y)
    return videoWidthPip/sourceRect.width
} //限制画中画拖动范围，返回背景视频的分辨率
}

//add 画中画窗口
Rectangle {
    // _pipPlay,画中画播放器实例化
    MediaPlayer {
        id: _pipPlay
        videoOutput: addOut
        onMediaStatusChanged: {
            if (mediaStatus === MediaPlayer.EndOfMedia) {
                addVideo.visible = false
            } //画中画播放完毕就消失画中画窗口
        }
    }
}
VideoOutput {
    id: addOut
    fillMode: VideoOutput.PreserveAspectFit // 保持比例
}

DragHandler {
    id: dragHandler
    target: addVideo
    //动态获得画中画的分辨率
    onActiveChanged: {
        if (!active) {
            scalePip = oneplayer.videoOutput.drag(addVideo)
        }
    }
}

//可拖动文本提示
Text {
    anchors {
        top: parent.top
        horizontalCenter: parent.horizontalCenter
        topMargin: 10
    }
}

```

```

    }
    text: "可拖动!"
  }
}

```

//画中画按钮

```

Button{
  text:"添加画中画"
  onClicked: {
    oneplayer.openAddV.open()
    addVideo.visible = true
  }
}
Button{
  text:"确认画中画位置并保存"
  //保存画中画视频的 FileDialog,在这里设置对话框是为了方便参数传递,但不美观
  FileDialog{
    id:_savepip
    onAccepted: {
      outputPathPip = selectedFile.toString().replace("file://", "")
      const fileName1 = selectedFile.toString().split('/').pop().replace(/\.[^/.]+$/, "")
      let pipVideoPath = _pipPlay.source.toString().replace("file://", "")
      let pipDuration = _pipPlay.duration/1000

```

```

pipper.videoInsertPip(inputPathPreview,pipVideoPath,outputPathPip,pipDuration,addVideo.x/
scalePip,addVideo.y/scalePip)

```

```

      oneplayer.savemodel.append({
        filepath:selectedFile,
        filename:fileName1
      })
    }
  }
}

```

画中画位置在预览窗口的预览效果大概是与输出文件的画中画效果是差不多的，但还可以再细化，确定它的输出画中画的大小，而不是固定的 1/4

3.8 界面底部存放处理好的视频

//通过 ListView 实现

```

ListView{
  id:saveVideoView
  anchors.fill:parent
  orientation: ListView.Horizontal
  spacing:5

  model:oneplayer.savemodel

  //delegate
}

```

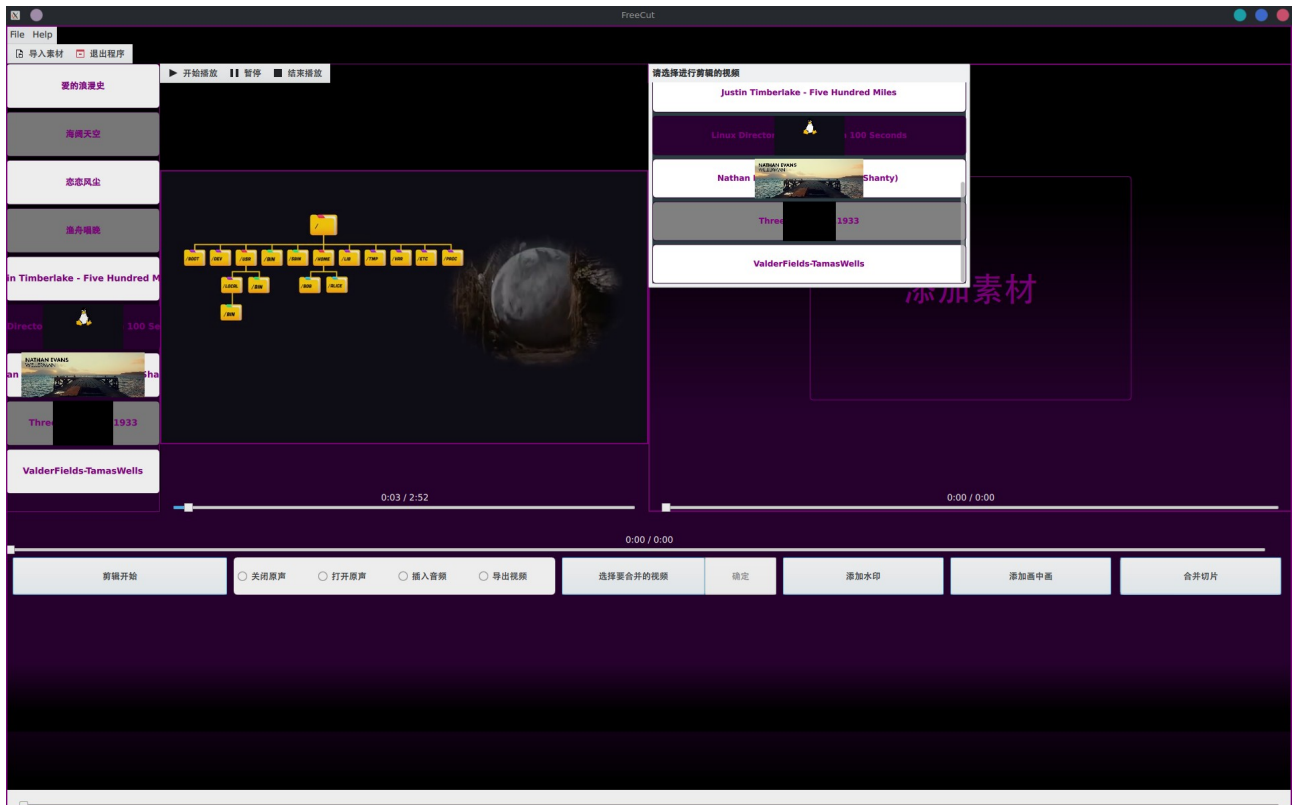
//所有最终视频保存的对话框都会有这部分代码
savemodel.append({

```
filepath:selectedFile,
filename:fileName1,
filepath:outputPath
})
```

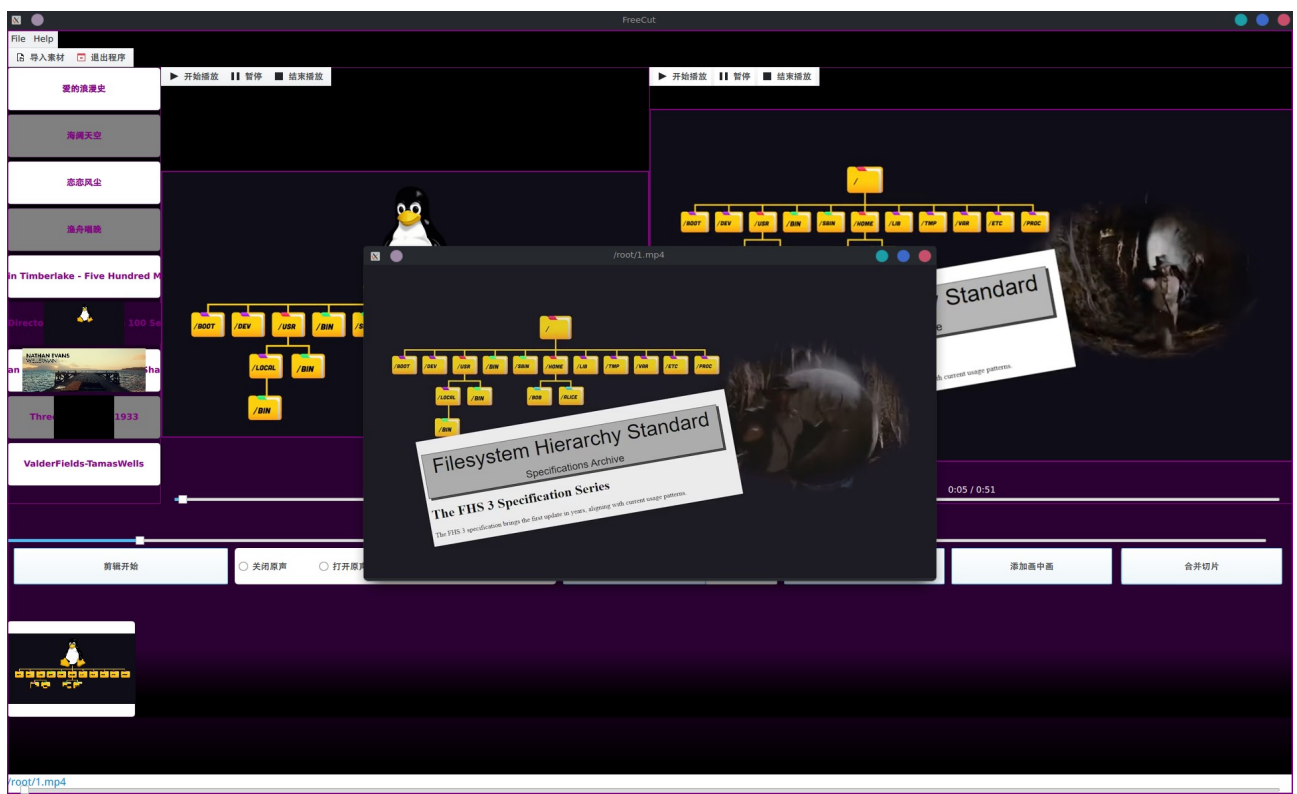
3.9 界面设计

主界面布局（分为菜单工具栏，最顶部，左边是素材导入列表，中间是素材播放窗口，右边是预览窗口，最下面是剪切等功能的实现和处理好的视频的存放窗口）

（图 1）



（图 2）



关键组件：

1. 素材列表：显示导入的媒体文件
2. 播放窗口：当前选中素材的播放区域
3. 预览窗口：编辑效果预览

4. 剪辑控制区：提供剪辑操作流程

- 剪辑开始 → 设置起点 → 设置终点 → 预览 → 保存|继续剪切（单击保存，双击继续剪切或者退出）
保存->保存成功，保存的视频会放到底部，如图 2 所示
继续剪切->重复剪辑的操作

-合并切片：可以在一个视频中多次剪切并合并这个切片

-合并切片 → 开始剪切 → 结束剪切 → 确定 → 合并（合并可以选择也可以不选择，如果只剪切一次之后就立马合并，那么这个合并就相当于一次剪切的功能）

如果想继续剪切切片就重复：“开始剪切 → 结束剪切 → 确定”的操作，直到选择合并为止，一个合并切片的功能流程就结束了

5. 特效操作区：

- 水印设置
- 画中画添加
- 视频合并
- 音频替换

3.10 视频处理流程

A[导入素材] --> B[素材列表]
B --> C[选择素材]
C --> D[播放窗口预览]
D --> E{选择操作}
E --> |剪辑| F[设置起止点]
E --> |水印| G[设置水印参数]
E --> |画中画| H[选择小视频]
F --> I[生成预览]
I --> J[保存结果]
J --> K[结果展示区]

4. 构建与运行

4.1 依赖项

- Qt 6.9+
- FFmpeg (包含 ffmpeg/ffplay)
- C++23 兼容编译器

4.2 构建

建议勾选 Desktop Qt 6.9.1

4.3 运行配置

CMakeLists.txt 关键配置

```
qt_add_executable(appVideoclips)
qt_add_qml_module(appVideoclips
    URI Videoclips
    VERSION 1.0
    SOURCES main.cpp
```

```
QML_FILES Main.qml Actions.qml Player.qml ...
SOURCES videoplay.cpp videocutter.cpp ...
)
target_link_libraries(appVideoclips PRIVATE Qt6::Quick)
```

4.4 软件安装包

适用于 arch linux（即定制系统的 x11，wayland 界面布局有点问题）

安装命令：pacman -U freecut-1.0-1-x86_64.pkg.tar.zst

5. 使用示例

5.1 视频剪辑

1. 点击"导入素材"添加视频
2. 在素材列表选择视频
3. 点击"剪辑开始"
4. 拖动时间轴设置起点 → 点击"选择该节点作为剪辑的第一个节点"
5. 设置终点 → 点击"选择该节点作为剪辑的第二个节点"
6. 预览效果 → 保存结果

5.2 添加水印

1. 点击"添加水印"
2. 在弹出框中设置：
 - 文本内容
 - 颜色/大小/透明度
3. 选择输出路径
4. 等待处理完成

5.3 画中画

1. 点击"添加画中画"
2. 选择背景视频
3. 选择小视频
4. 拖拽调整位置
5. 点击"确认画中画位置并保存"

6. 已知问题与优化方向

当前限制：

1. 部分视频格式兼容性问题
2. 大文件处理时内存占用较高且处理过程中只是暂停了预览窗口的播放，没有“正在处理”的提示
3. 复杂编辑操作缺乏撤销功能
4. 界面还可以优化的更系统专业
5. 存有不同平台的兼容性问题和分辨率问题，相同的代码在不同平台，如 wayland 等运行界面不同

优化方向：

- GPU 加速视频处理
- 多轨道时间线支持
- 转场效果添加
- 视频导出质量设置
- 动态调整布局以更灵活的适应不同图形界面系统

7. 团队成员以及负责的工作概述（详见工作日志）

陈敏：剪切功能。画中画功能。预览功能（弹出窗口）。软件打包。问题优化。

王婷：水印功能。多次剪切的逻辑。视频进度条（时间轴）的实现。ui 交互优化。

郭瑞桦：确定了基本 ui 界面框架。列表素材导入实现。原声开关+替换音频。合并+合并切片。
指导老师：龚伟 stardust <open-src@qq.com>

软件安装包：

最新版本: v1.0

更新日期: 2025-07-09

开发团队: CHW (@qq.com)